

OpenClaw on an 8 GB CPU host

A step-by-step installation with Docker, Ollama and Discord

Target: a fresh Ubuntu 24.04 LTS computer, Intel/AMD x86-64 (**amd64**), 8 GB total RAM, no GPU, about 15 GB free disk, internet access, and a normal user with **sudo** access. You also need your own Discord account and permission to add a bot to a server.

Result: one local text-and-image agent, a restricted Discord channel, and editable persona and file-handling instructions. Allowed users share the conversation and files. Audio and video are outside this procedure.

Component	Pinned selection
OpenClaw	2026.9.8 with its official Discord plugin
Ollama	0.40.0
Model	qwen3.5:2b-q4_K_M, text and images
Memory caps	Ollama: 4 GiB; OpenClaw: 2 GiB

The model download is approximately 1.9 GB; this is not its RAM usage. Whole-host fit on 8 GB remains unmeasured. Complete step 15 before accepting this deployment. Inference is local; Discord transports messages and images.

1. Check your computer

Open Terminal as your normal Linux user. Run each block in order and stop on an error. Preserve newlines: a trailing `\` continues a command.

```
cat /etc/os-release
dpkg --print-architecture
free -h
df -h "$HOME"
```

Check: Ubuntu 24.04, architecture **amd64**, and sufficient RAM and disk.

2. Install the required host tools

```
sudo apt-get update
sudo apt-get install -y ca-certificates curl git nano \
python3 procps ripgrep openssh-client
```

sudo asks for your Linux password. Docker will supply OpenClaw, Ollama, Node.js and npm; they do not need separate host installations.

Install Docker and download the project

3. Install Docker Engine and Compose

These commands target a fresh Ubuntu 24.04 amd64 system. If Docker, containerd or Podman is already installed, check the package conflicts in the official Docker instructions first; do not remove a runtime serving other workloads.

```
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
sudo tee /etc/apt/sources.list.d/docker.sources >/dev/null <<'EOF_DOCKER'
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: noble
Components: stable
Architectures: amd64
Signed-By: /etc/apt/keyrings/docker.asc
EOF_DOCKER
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
sudo systemctl enable --now docker
```

4. Enable Docker for your user and verify it

Docker group membership grants administrative control of this computer. Add only your trusted operator account.

```
sudo groupadd -f docker
sudo usermod -aG docker "$(id -un)"
```

Log out completely and log back in, or reconnect your SSH session. Then run without `sudo`:

```
id -nG
docker version
docker compose version
docker run --rm hello-world
```

Check: The group list includes `docker`; Client and Server respond; Compose is v2 or newer; the test prints Hello from Docker!

5. Download Sonne

No GitHub account or token is required.

```
mkdir -p ~/agents
cd ~/agents
git clone https://github.com/Krabbens/sonne.git
cd sonne
ls compose.yaml .env.example scripts/configure.py
```

Check: All three files are listed. Run later terminal commands from this folder. After reopening Terminal, use `cd ~/agents/sonne`.

Create your Discord bot

6. Create the application and invite the bot

In your browser:

1. Open `discord.com/developers/applications`. Choose **New Application**; name it **Sonne**.
2. Open **Bot**. Enable **Message Content Intent**. Leave Server Members and Presence intents disabled.
3. Generate or reset the bot token. Keep it for step 8; do not publish it.
4. Open **OAuth2 / URL Generator**. Select scopes `bot` and `applications.commands`.
5. Grant **View Channels**, **Send Messages**, **Read Message History**, **Embed Links** and **Attach Files**. Do not grant Administrator.
6. Open the generated invitation URL, choose your server, and authorize the bot.
7. Create one ordinary text channel accessible to the bot and intended users.

Check: Your own bot appears in your server. It may stay offline until step 11.

7. Copy the four types of ID

1. Copy the **Application ID** from the application's General Information page.
2. In Discord, open **User Settings / Advanced**; enable **Developer Mode**.
3. Right-click your server and select **Copy Server ID**.
4. Right-click the selected channel and select **Copy Channel ID**.
5. Right-click each permitted user and select **Copy User ID**.

Use numeric IDs, not names. Direct messages and other channels are disabled by the supplied configuration.

8. Enter your token and IDs

Back in Terminal:

```
cd ~/agents/sonne
cp .env.example .env
nano .env
```

Replace these five settings. The digits below are examples only.

```
DISCORD_BOT_TOKEN=your-real-bot-token
DISCORD_APPLICATION_ID=111111111111111111
DISCORD_GUILD_ID=222222222222222222
DISCORD_CHANNEL_ID=333333333333333333
DISCORD_USER_IDS=444444444444444444,555555555555555555
```

Separate user IDs with commas and no spaces. Leave gateway token and local UID/GID blank; keep the image settings unchanged. Save in nano with **Ctrl+O**, **Enter**, then exit with **Ctrl+X**.

9. Generate and check the configuration

```
python3 scripts/configure.py
docker compose config --quiet
```

Check: Both commands exit successfully. The generator creates private state, sets your container UID/GID and generates the gateway token. Run it without `sudo`.

Download the model and start the agent

10. Pull the images and install the Discord plugin

```
docker compose pull
```

Wait for both images to finish downloading. Install the pinned official plugin:

```
docker compose run --rm --no-deps openclaw \
  node dist/index.js plugins install @openclaw/discord@2026.9.8 \
  --pin --accept-capabilities --force
```

This is the first-install command; `-force` can overwrite an existing plugin. Then validate:

```
docker compose run --rm --no-deps openclaw \
  node dist/index.js config validate --json
```

Check: Validation succeeds with no missing-plugin or unfinished-migration warning. If it fails, stop and use the diagnostics on page 6.

11. Download the exact model and start OpenClaw

```
docker compose up -d ollama
docker compose exec -T ollama ollama pull qwen3.5:2b-q4_K_M
docker compose exec -T ollama ollama list
```

Check: The list contains `qwen3.5:2b-q4_K_M`. Do not use the bare `qwen3.5` tag; it selects a larger model.

```
docker compose up -d openclaw
docker compose ps
```

Check: Both services become healthy. Health alone does not prove inference works.

12. Check the local model and Discord connection

Run the model-routing probe:

```
docker compose exec -T openclaw node dist/index.js \
  infer model run --local --agent sonne \
  --model ollama/qwen3.5:2b-q4_K_M \
  --prompt 'Reply with exactly: sonne-ok' --json
```

Check: The completion contains `sonne-ok`. The first CPU request can be slow.

Now open the allowed Discord channel as an allowed user. Mention the actual bot:

```
@Sonne Reply with exactly: sonne-ok
```

Check: A visible reply arrives in Discord. This checks the agent connection; the terminal probe checks only model routing.

If either check fails:

```
docker compose logs --tail=100 openclaw
docker compose logs --tail=100 ollama
```

Set the prompt and test useful work

13. Customize the supplied instructions

The repository already includes an English persona in `SOUL.md` and file-handling rules in `AGENTS.md`. Edit the host copies:

```
nano templates/workspace/SOUL.md
nano templates/workspace/AGENTS.md
```

The supplied persona begins:

```
You are Sonne, a practical assistant for understanding images and
working with small text files. Reply in the user's language.
Keep answers concise and concrete.
```

```
Describe what is visible in an image, including relevant text.
Distinguish observations from guesses. If text is unreadable,
ask for a closer crop. Never claim to see an image that was not
provided or successfully loaded.
```

Keep the rules requiring real tool results, workspace-only file access and confirmation before overwriting. These files supplement OpenClaw's internal system prompt. Apply your changes:

```
docker compose up -d --force-recreate openclaw
```

Start a fresh session in the optional operator UI at `127.0.0.1:18789`; enter the gateway token from `.env`. Pair your browser if requested (see `README`). Prompt files are read-only inside the container. Only `/workspace/files` is writable; it maps to host `workspace/files/`.

14. Verify images, files and access restrictions

Perform these checks in the allowed channel, one request at a time:

1. Attach `examples/vision-check.png`. Mention the bot and ask: *Read the large label and identify the red shape*. Expect **SONNE 42** and a **red square**. Repeat with your own readable screenshot.
2. Ask the bot to create and read back a disposable file:

```
@Sonne Create files/smoke.txt containing exactly SONNE FILE OK.
Read it back and report the workspace-relative path.
```

Verify on the host:

```
cat workspace/files/smoke.txt
```

3. Ask to overwrite that file. Require confirmation, approve, then check the file again.
4. Ask to read `/etc/passwd` and edit `SOUL.md`. Verify denied tool results in session details or logs, not just a verbal refusal.
5. Try a DM, another channel and an unlisted user. Expect no agent reply or file action.

Check: The actual image and file results match. All permitted users share the files.

Accept the deployment and keep it running

15. Measure memory under representative load

In separate terminals, monitor while repeating step 14, including a cold model load and requests from two users:

```
free -h
docker compose stats
vmstat 1
docker compose exec -T ollama ollama ps
```

Confirm CPU processing. In `vmstat`, ignore the first row and inspect later `si/so` columns for sustained swap activity. After the test:

```
docker inspect $(docker compose ps -aq) \
  --format '{{.Name}} OOM={{.State.OOMKilled}} restarts={{.RestartCount}}'
sudo journalctl -k --since '15 minutes ago' | rg -i 'oom|out of memory'
docker compose restart
cat workspace/files/smoke.txt
```

Check: Tasks complete, no OOM kill or repeated restarts, available host RAM stays above 512 MiB, no continuous swap growth, and files survive restart. Record CPU, usable RAM, memory peaks and response times. If these fail, this host is not accepted.

The preset uses a 65,536-token context, 1,024-token output cap, no GPU, thinking off and serialized work. A 2B model can misread text or misuse tools; check real outputs. The 64k context increases memory demand; repeat this check under the existing caps.

Routine commands

```
docker compose logs --tail=100 openclaw
docker compose restart openclaw
docker compose stop
docker compose start
```

If startup fails. Check token, numeric IDs, Message Content Intent, channel permissions and an explicit mention. After editing `.env`, rerun `python3 scripts/configure.py` and recreate OpenClaw. For unfinished plugin migrations, inspect `node dist/index.js update status -json` with the same `docker compose run -rm -no-deps openclaw` prefix as step 10. For Docker Desktop mount errors, update the checkout and follow README's mount check.